

DAG tokenizer using greedy sem cov

yuanyuan.zheng

August 2026

1 Introduction

Electronic health record (EHR) foundation models (FMs) have recently emerged as promising assistive tools for clinical decision support [CITE: recent EHR foundation model paper(s), e.g. a transformer-based EHR-FM trained on longitudinal patient trajectories]. Because clinical events are recorded heterogeneously across institutions, such models are typically trained over standardized clinical vocabularies – ICD, RxNorm, SNOMED CT, LOINC – rather than free text, so that the same clinical fact is represented the same way regardless of where it was recorded [CITE: standardized clinical vocabulary / interoperability reference]. Several of these vocabularies are not flat code lists but ontologies structured as directed acyclic graphs (DAGs), which is what lets them normalize clinical naming while also supporting interoperability across terminologies. SNOMED CT is the clearest example: it comprises more than 400,000 concepts covering disorders, procedures, medications, morphologic findings and more, connected by over 100 relationship types that let each concept be expressed as a machine-readable logical composition of others, and it is mappable to most other major clinical terminologies [CITE: SNOMED International concept model / release statistics].

Training an EHR FM directly on top of such a vocabulary creates a tension between two resources that are each expensive on their own. Learning a reliable representation for every concept in the vocabulary requires a correspondingly large amount of patient data, which is costly to obtain and often access-restricted [DOUBT: do you want to cite/quantify this, e.g. typical cohort sizes needed vs. available, or leave it as a general statement?]. Restricting training to a vocabulary small enough to be learnable from the data actually at hand is not free either, precisely because clinical vocabularies are large by construction and concept usage in real EHR data follows a long tail: most concepts occur too rarely for a model to learn a stable representation for them [CITE: long-tail / rare-code distribution in clinical NLP or EHR modeling].

Existing strategies for shrinking the effective vocabulary size largely leave this semantic structure on the table. Data-driven subword approaches, such as byte-pair encoding applied to categorical medical codes, merge concepts purely

by co-occurrence statistics: they preserve no explicit semantics and the resulting vocabulary is highly sensitive to data quality and to the order in which concepts are seen [CITE: byte-pair-style tokenization of medical codes]. Frequency- or entropy-based top- k selection keeps only the most prevalent, or highest-information-contributing, concepts, discarding a considerable amount of clinically relevant but rarer information [CITE: prevalence/entropy-based vocabulary pruning in clinical ML]. Truncating or rolling leaf concepts up to their ontology parents reduces vocabulary size at the direct cost of clinical granularity – a specific diagnosis collapses into a broader category [CITE: ontology roll-up / ICD truncation approach]. Keeping every concept exactly as-is sidesteps these losses only when enough patient data is available, and otherwise simply defers the long-tail problem to the model [DOUBT: is there a citation you have in mind for this "no reduction" baseline, or is it meant as the trivial comparison point rather than a cited method?]. Finally, learnable embedding-space codebooks (e.g. vector-quantization-style tokenizers) can compress the vocabulary without hand-designed rules, but at the cost of controllability and interpretability: the resulting tokens no longer correspond to identifiable, auditable clinical concepts [CITE: learnable codebook / VQ-style tokenization]. [DOUBT: Is the sharpest way to state the gap "none of these use the DAG's semantic structure at all," or the stronger claim "none of these produce a token set that is both semantically motivated *and* inspectable/controllable"? The second framing lines up more directly with the contribution list below – your call on which to lead with.]

We propose a token-selection and tokenization method that operates directly on a concept DAG such as SNOMED CT. First, we select a small set of tokens using a greedy, submodular semantic-coverage objective (Section 3), which fixes the target vocabulary size in advance rather than as a side effect of a training run. Second, rather than representing a concept as an unordered bag of the tokens it maps to, we represent it as a *context tree*: a small subgraph that preserves which relationship each token was reached through, so that, for instance, a token for "kidney" and a token for "left" stay attached to the same anatomical-site branch instead of being silently merged with an unrelated "right adrenal gland" branch found elsewhere in the same concept [DOUBT: this kidney/adrenal example is carried over from your existing PDF draft – keep it, or swap in a real example from the HUG-mapped concepts once we have one handy?]. Third, applying this method to the SNOMED CT concepts mapped by our institution's semantician team, we show that, unlike the alternatives above, the resulting tokenization is directly controllable and interpretable: for any given concept, a domain expert can inspect exactly which relationships were followed to reach which token. [NOTE: Your outline's contribution item 5 – "if any mapping inter-ontology exists, we can use concepts in one ontology to tokenize concepts in another" – is repeated almost verbatim in the Future Work section (item 1). I left it out of this paragraph to avoid stating it twice; see Section 7 for the one place I kept it, and tell me if you'd rather list it here instead (e.g. because you already have a toy cross-ontology

demonstration) and drop it from Future Work.]

2 Background

2.1 Directed acyclic graphs

A directed acyclic graph (DAG) is a directed graph $G = (V, E)$ that contains no directed cycle: there is no sequence of edges that both starts and ends at the same vertex while always following edge direction. Equivalently, G admits at least one topological ordering of V in which every edge points from an earlier to a later vertex in the ordering. Acyclicity is what makes "child" and "parent" well defined for any two vertices connected by a directed path: a vertex reached by following edges forward is strictly more specific than, and can never be an ancestor of, the vertex it was reached from.

2.2 SNOMED CT as a semantic DAG

SNOMED CT is one such graph. It comprises $|V| \approx 400,000$ concepts [CITE: SNOMED International release statistics] spanning disorders, procedures, findings, body structures and substances, connected by [RESULTS NEEDED: exact number of distinct relationship types in the current release / extension in use] relationship types. Two properties of this graph matter for the rest of the paper.

First, it is acyclic. [RESULTS NEEDED: confirm this holds for the current working subgraph (combined_subgraphs, built at max_dist_candidate = 9) the same way it held for the earlier, smaller $D = 3$ subgraph – the tokenizer’s own ISA-transparency rule (Section 3) and several of the baselines (Section 3) implicitly rely on well-defined, finite hop distances, which acyclicity guarantees.]

Second, its edges carry semantic value beyond hierarchy. One relationship type, `IS_A`, encodes generalization/specialization – the parent/child relation defined above. Every other relationship type (for example *finding site*, *associated morphology*, *has active ingredient*) attaches an attribute to a concept by pointing to another concept, rather than by storing a value locally. As a consequence, a concept’s meaning is not contained in the concept itself but constructed from its position and connections inside the DAG – two concepts that look unrelated by name can share most of their meaning if they are reached through the same relationships to the same targets, and conversely two occurrences of the same relationship type from the same concept can point to entirely different, non-interchangeable aspects of it. This is exactly the structure our method is built to exploit: Section 3 defines a coverage score and a tree representation that treat `IS_A` edges and non-`IS_A` edges differently for precisely this reason.

2.3 Working vocabulary and notation

Let $M \subseteq V$ denote the subset of SNOMED CT concepts mapped by our institution’s semantic team to locally used clinical concepts ($|M| \approx 40,000$), and let $T \subseteq V$, $|T| = k$, denote a *token set*: a small subset of concepts chosen to represent every concept in M (Section 3). Candidate tokens are not drawn from the whole of V but from a bounded neighborhood of M ; the exact candidate pool and how it is obtained is described together with the method in Section 3, since the same depth budget D that defines the candidate pool also defines the tokenization horizon.

3 Method

3.1 Candidate pool and depth budget

A single depth budget D plays three roles in this method, all controlled by the one hyperparameter `TokenizerParam.max_dist_candidate` in the implementation: it bounds how far a candidate token may be from the mapped concepts it can help represent, it bounds how deep a context tree (Section 3.3) is allowed to grow, and it bounds the horizon over which the coverage score below is computed. Concretely, the candidate pool is

$$V_D = M \cup \{u \in V : u \text{ is reachable from some } c \in M \text{ by at most } D \text{ relationships}\},$$

obtained in practice as the union of the D -hop ego-graph (following outgoing edges) around every mapped concept. Token selection (Section 3.2) only ever considers $t \in V_D$: a candidate that no mapped concept can reach within D hops has zero possible contribution to the objective below and is never evaluated. **[DOUBT: Using one shared D for all three roles is a simplifying design choice worth calling out explicitly – a reviewer may ask why the candidate radius, the tree depth, and the selection horizon are tied together rather than treated as three independent hyperparameters. Is that coupling intentional/load-bearing, or just a current implementation convenience?]**

3.2 Semantic coverage and greedy token selection

For $u \in V$, $T \subseteq V$ and horizon $h \in \{0, \dots, D\}$, let $S_h(u, T) \in [0, 1]$ denote how well u is covered by T when at most h further relationships may be traversed. The base case is

$$S_0(u, T) = \mathbf{1}[u \in T],$$

and, extending the recursion with a per-hop distance penalty $\lambda \in (0, 1]$,

$$S_h(u, T) = \begin{cases} 1, & u \in T, \\ 0, & u \notin T \text{ and } d^+(u) = 0, \\ \frac{\lambda}{d^+(u)} \sum_{(u,r,v) \in \text{Out}(u)} S_{h-1}(v, T), & \text{otherwise.} \end{cases} \quad (1)$$

A token is absorbing (its coverage is 1 at every remaining horizon); every other node’s coverage is the λ -discounted average coverage of its out-neighbors. Setting $\lambda = 1$ recovers the undiscounted score, in which a token found j hops away and a token found at the root contribute equally; for $\lambda < 1$, a token found j hops away instead contributes λ^j , so the score also rewards *proximity* to a token, not only reachability. The overall objective is the mean coverage of the mapped concepts at the full horizon D :

$$F_D(T) = \frac{1}{|M|} \sum_{c \in M} S_D(c, T), \quad F_D(\emptyset) = 0, \quad 0 \leq F_D(T) \leq 1.$$

F_D is normalized, monotone, and submodular in T (the marginal gain of adding a token never increases as more tokens are already selected), which follows from writing F_D as a positive average, over random depth- D walks from each $c \in M$, of the indicator that the walk hits T . Submodularity gives the greedy algorithm the classical approximation guarantee

$$F_D(T_k^{\text{greedy}}) \geq \left(1 - \frac{1}{e}\right) \max_{|T| \leq k} F_D(T),$$

and, more importantly for a candidate pool of tens of thousands of nodes, lets us use *lazy* greedy instead of exact greedy without changing the selected set: each candidate’s previously computed marginal gain is a valid upper bound on its current gain, so at every step it suffices to re-verify only the best stale candidate against the next-best stale bound, rather than recomputing every remaining candidate. Adding one candidate t only changes S_h at t itself and at vertices that can reach t within D hops in reverse, so a single candidate’s exact marginal gain can be computed by propagating a sparse delta backward through the incoming-edge structure instead of recomputing the full $O(D(|V| + |E|))$ recursion from scratch.

3.3 Tokenization via context trees

Once a token set T is fixed, a mapped concept $c \in M$ is not represented by the flat, unordered set of tokens reachable from it. A flat representation throws away exactly the information that non-IS_A relationships were designed to carry: if a concept has two independent *finding site* occurrences, each qualified by its own *laterality*, a flat set of tokens {kidney, adrenal gland, left, right} can no longer tell which laterality belongs to which site. **[DOUBT: keeping this exact**

[FIGURE PLACEHOLDER – illustration of the λ -decayed coverage propagation, e.g. the small worked example from your PDF ($u \rightarrow \{t_1, v\}, v \rightarrow \{t_2, z\}$): show S_h propagating backward from t_1, t_2 toward u , and how the contribution shrinks by a factor of λ per hop.]

Figure 1: Semantic coverage propagation (to be replaced with an actual figure/diagram).

kidney/adrenal example from your PDF – confirm, or swap for a real HUG-mapped concept once we pick an illustrative one for the figure below.]

We instead recursively expand each mapped concept c into a *context tree*. **IS_A** relationships are transparent and continue the search in the current context; every other relationship type opens a fresh subcontext, so that two independent occurrences of the same non-**IS_A** relationship type are kept in two separate subcontexts. Formally, $\text{EXPAND}(u, q, d)$, starting from $\text{EXPAND}(c, q_0, 0)$:

1. if $u \in T$, attach u as a token of the current context q and stop this branch;
2. otherwise, if $d = D$ or u has no outgoing edges, mark this branch *uncovered* in q and stop;
3. otherwise, for every outgoing edge (u, r, v) : if $r = \text{IS_A}$, continue in the same context q ; otherwise, open a new subcontext of q labelled by r and continue there.

A token located exactly at depth D is still accepted (case 1 is checked before case 2). The resulting tree keeps only T as its vocabulary; its subcontext structure records exactly which relationship chain each token was reached through, which is what preserves the kidney/left vs. adrenal gland/right distinction above.

[FIGURE PLACEHOLDER – side-by-side: flat token set vs. context tree for a real mapped concept with ≥ 2 non-**IS_A** relationship occurrences, e.g. the kidney/adrenal-gland example, or a concept pulled directly from the HUG-mapped data.]

Figure 2: Context-tree tokenization example (to be replaced with an actual figure).

3.4 Evaluation metrics

Every metric below is computed once the context trees for all of M have been built against a fixed T ; they evaluate the *realized* representation, as opposed to $F_D(T)$, which is the objective used to *select* T in the first place (Section 3.2). We group them by what aspect of the tokenization they capture:

Efficiency – how compact the representation is. *Conciseness* is the mean number of token leaves per concept’s tree; *tree complexity* is the mean number of contexts (root plus subcontexts) per tree. Lower is better for both: a concept represented by fewer tokens and fewer subcontexts is cheaper to encode and easier to inspect.

Fidelity – how close, on average, the tokens actually found are to the concept they represent. *Distance score* converts the mean hop-distance at which tokens are found (uncovered branches counted at $D + 1$ rather than dropped) into a $(0, 1]$ similarity, $1 - \bar{d}/(D + 1)$. Higher is better: 1.0 means every branch found a token immediately.

Discriminative power – how much the token vocabulary lets distinct concepts remain distinguishable from one another. *Uniqueness entropy* is the normalized Shannon entropy of context-tree signatures across M : it is low when many concepts collapse onto the same tree shape (i.e. become indistinguishable under T) and high when signatures are spread out. *Unique rate* is its per-concept decomposition – the fraction of concepts whose own signature is not shared by any other concept. Higher is better for both.

Semantic content – how much of each concept’s meaning was captured at all. *Semantic coverage* is $F_D(T)$ itself, always reported at $\lambda = 1$ regardless of the λ a candidate list was *selected* under **[DOUBT: confirm this is still the evaluation convention you want – i.e. selection sweeps λ , but the reported metric always scores coverage at $\lambda = 1$, so every method is compared on the same, undiscounted notion of “did it reach a token at all”]**. *Uncovered rate* is the fraction of concepts with *zero* tokens anywhere in their tree – not represented at all. *Unk rate* is the (weaker) fraction of concepts with *at least one* uncovered branch somewhere in their tree, i.e. partially unrepresented; by construction uncovered rate $\leq$ unk rate. Higher is better for semantic coverage; lower is better for both uncovered rate and unk rate.

[NOTE: Your outline’s four families (efficiency / fidelity / discriminative power / semantic content) account for 7 of the 8 metrics `eval.py` actually computes. The eighth, exact rate – the fraction of mapped concepts that are themselves selected as a token, i.e. a 0-hop / exact representation, higher is better – isn’t placed anywhere in your grouping. I’ve provisionally added it under Semantic content below, since a 0-hop match is the ceiling case of coverage, but it could just as naturally sit under Efficiency (the most concise possible representation) or get its own “direct representation” family – your call.] Finally, *exact rate* is the fraction of mapped concepts that are themselves selected as a token (a 0-hop, exact representation); higher is better, and it is the ceiling case both of semantic coverage and of conciseness.

3.5 Baselines

All baselines below rank the full candidate pool V_D once and are then truncated with the top- k prefix of that ranking, exactly as the greedy method’s own selection history is truncated – so every method in Section 4 is compared at matched

k .

Random. *k-random*: for each k , an independent uniform sample of k candidates without replacement, repeated over multiple seeds and averaged, since it is not a single ranking that can be truncated the same way as the others.

Degree-based. *Highest degree*: rank by number of distinct predecessors within D hops, over every relationship type. *Highest degree, $\Delta \leq 1$* : the same score restricted to direct (1-hop) predecessors. *Most children*: the same D -hop predecessor count, but restricted to the IS_A-only hierarchy edge subgraph – i.e. how many descendant concepts a candidate has in the hierarchy alone, ignoring every other relationship type.

Centrality-based. *PageRank*: the stationary distribution of a random walk with restart probability $1 - \alpha$ ($\alpha = 0.85$) [CITE: Brin & Page, PageRank]. *Personalized PageRank*: the same recursion but restarting uniformly on M instead of on all of V , making it the one centrality baseline that is M -aware rather than purely structural [CITE: personalized / topic-sensitive PageRank, e.g. Haveliwala]. *Closeness centrality*: the (in-degree-oriented, Wasserman–Faust improved) reciprocal of the mean shortest-path distance to a candidate from every node that can reach it [CITE: Freeman, closeness centrality]. [NOTE: Your outline’s centrality-based list (item 4.2) names only these three. The code (5.3.centrality_baselines.ipynb, still referenced by the current config.CandidateLists) also computes a fourth: eigenvector centrality. I’ve added it below – drop it if you’ve decided against reporting it.] *Eigenvector centrality*: rank by the left eigenvector of the adjacency matrix for its largest-modulus eigenvalue [CITE: eigenvector centrality / Bonacich]. [DOUBT: worth flagging to reviewers explicitly: the uniqueness guarantee for this measure (Perron–Frobenius) only holds on a strongly connected graph, which a DAG by definition is not – on the earlier, smaller candidate subgraph this baseline was observed to collapse almost all candidates to a near-zero score, making its ranking past the first few thousand candidates close to an arbitrary tie-break. Please re-verify (and, if still true, quantify) this on the current $D=9$ candidate pool before we decide how much weight to give this baseline in Section 4.]

Ours, and its hard-greedy ablation. Our method is the lazy-greedy maximizer of F_D from Section 3.2. As an ablation, we also report a *discrete set-cover* selector that keeps the same lazy/CELF greedy mechanism – and the same $(1 - 1/e)$ approximation guarantee – but replaces the soft, λ -decayed, multi-hop-averaged coverage $S_h(u, T)$ with a binary “is c within D hops of some token in T ” indicator, i.e. classical maximum-coverage greedy applied to the same candidate pool. Comparing the two isolates what the soft, distance-aware formulation of Eq. (1) buys over textbook hard-coverage greedy, rather than comparing against a strawman.

[NOTE: Your outline’s item 5, “(maybe) evaluation of acceptability of the tokenized version vs. the original concept, by a clinical expert,” is also referenced by Limitations item 3 (“no external validation on a clinical task”) and Future Work item 2 (“external validation clinical task”). Right now the three don’t

agree on its status – please pick one: either it’s an expert-rating study that happens in this paper (belongs here, and Limitations item 3 needs softening), or it’s genuinely future work (drop it from this Method list entirely and keep it only in Future Work).]

4 Results

4.1 Data and candidate pool

We use every SNOMED CT concept mapped by our institution’s semantician team in HUGData as the mapped set M . *[NOTE: "HUGData" is used here exactly as it appears in your own `config.py` path names – flag if this should instead be spelled out or anonymized for the paper.]* Candidate tokens are restricted to the D -hop neighborhood of M (Section 3.1), which narrows the pool from $|V| \approx 400,000$ SNOMED concepts to **[RESULTS NEEDED: current candidate-pool size under `max_dist_candidate = 9`]**. **[DOUBT: Your outline states this narrowing as "400k -> 70k," which matches the 75,634-node figure from the old (obsolete) $D = 3$ analysis – not the current $D = 9$ setting. A larger D grows the ego-graph radius substantially, so the current pool is very likely much larger than 70k, possibly a large fraction of all of V . Please recompute this from a current run of `1.graph_prepare.ipynb` before this number goes in the paper.]**

4.2 General trend across k

[RESULTS NEEDED: plot(s) of each metric from Section 3.4 against k , one line per candidate-selection method, at a fixed reporting λ (presumably $\lambda = 1$, consistent with Section 3.4’s evaluation convention). Send me the underlying table/plot and I will write the narrative text around it (not just "Figure X shows Y" but why the shape looks that way given the method).]

4.3 Comparison across baseline families

For each family from Section 3.5 – random, degree-based, centrality-based, and ours (with its hard-greedy ablation) – we plot the upper envelope of metric performance across the family’s members, so that the comparison is against each family’s *best* member rather than diluted by its weaker ones.

[RESULTS NEEDED: once the sweep is in, this subsection should state, per family, which member forms the envelope at which k , and characterize *why*. Below are hypotheses that follow from the method design in Section 3, phrased so they’re easy to confirm or reject against the actual numbers rather than asserted as findings:]

- [DOUBT: Degree-based methods (especially *most children*, which only looks at the IS_A subgraph) may tend to select candidates whose neighborhoods overlap heavily – e.g. several siblings under the same popular parent – which would show up as lower discriminative-power metrics (uniqueness entropy / unique rate) than a coverage-aware method at the same k , even where raw semantic coverage is comparable. Worth checking directly rather than assuming.]
- [DOUBT: Centrality-based methods (PageRank family, closeness, eigenvector) rank by global random-walk or shortest-path structure rather than by M -specific coverage; personalized PageRank is the one exception (it restarts on M). This predicts personalized PageRank should track our method more closely than plain PageRank or closeness does – again, a hypothesis to check, not yet a result.]
- [DOUBT: eigenvector centrality’s DAG degeneracy (flagged in Section 3.5) predicts its envelope contribution should flatten out once k passes whatever fraction of the pool actually carries non-negligible score – if that’s what the current run shows, it’s worth stating explicitly as an explanation rather than leaving the flat curve unexplained.]

4.4 Effect of the distance penalty λ

The degenerate case $\lambda = 0$. This case can be characterized exactly from Eq. (1) itself, independent of any particular run: at $\lambda = 0$, every term of the “otherwise” branch vanishes, so $S_h(u, T) = S_0(u, T) = \mathbf{1}[u \in T]$ for *every* horizon h , not only $h = 0$ – coverage never propagates. Substituting into F_D ,

$$F_D(T)|_{\lambda=0} = \frac{1}{|M|} \sum_{c \in M} \mathbf{1}[c \in T] = \text{exact_rate}(M, T).$$

In other words, selecting tokens at $\lambda = 0$ is not “coverage selection with a poor setting” but optimization of a *different* objective entirely: how many mapped concepts the selector picked as tokens of themselves. Any candidate not itself in M has zero marginal gain under this objective, so the selector has no signal to rank non- M candidates at all, and its behavior past the point where M is exhausted is effectively arbitrary tie-breaking. *[NOTE: This derivation only needs Eq. (1) and the definition of exact_rate (Section 3.4) – it holds regardless of which run’s numbers eventually back it up, so I’m confident stating it as a proposition rather than an empirical observation. [RESULTS NEEDED: the actual $\lambda = 0$ curve, to show this pathology concretely rather than just algebraically.]]*

Trade-off across $\lambda \in (0, 1]$. [RESULTS NEEDED: once available: how each metric family from Section 3.4 moves as λ increases from

0 toward 1. A reasonable prior, to be confirmed or corrected by the actual sweep, is that metrics tied directly to reaching *any* token (semantic coverage, unk rate, uncovered rate) should improve monotonically as $\lambda \rightarrow 1$ (evaluation is always at $\lambda = 1$, so selection and evaluation become better aligned), while metrics sensitive to *how far* a token was found (distance score, conciseness) may instead peak at some intermediate λ , since $\lambda = 1$ selection has no preference for nearby tokens over merely reachable ones. [DOUBT: state as hypothesis or confirmed finding depending on what the fresh sweep shows – do not carry over the specific numbers from the old, obsolete analysis even if the qualitative shape turns out to match.]]

Token reuse rate. [DOUBT: your outline flags this metric as "need to be implemented" – confirmed: `eval.py` does not currently compute anything under this name. This subsection stays a placeholder until that metric exists in code; happy to help define/implement it once you specify what "reuse" should mean here (e.g. how many distinct mapped concepts a single token ends up representing, across the whole context-tree population).]

Practical takeaway. [DOUBT: your outline's closing point, "best config depends on personal choice" – worth turning into one concrete sentence once the trade-off curves exist, e.g. naming which λ range is a reasonable default and what a practitioner would trade away by moving off it in either direction.]

[NOTE: Your PDF's own Section 7 ("Experiments") also proposes sweeping D itself – $D \in \{1, 2, 3, 4, 5\}$, measuring $|V_D|$ growth, reachable relationship counts, and degree distributions – as a separate axis from k and λ . That sweep doesn't appear anywhere in this Results outline, which otherwise treats D as fixed. Worth deciding explicitly whether a D -sensitivity analysis belongs in this paper's Results or is out of scope.]

5 Discussion

[DOUBT: This section has no content yet – your outline marks it "Empty for the moment," so nothing here is drafted as prose. Once Section 4 has real numbers, this section should close the loop back to Section 1's motivation. Suggested threads to pick up:]

- How much smaller a vocabulary does structured (greedy / degree / centrality) selection buy over the "keep everything" or "random" baselines from the Introduction's gap paragraph, at what cost in semantic coverage or distance score – i.e. turn Section 4.2/4.3 into a direct answer to "does this solve the FM vocabulary-size problem better than the alternatives listed in the Introduction."
- What the $\lambda = 0$ result (Section 4.4) implies practically: since λ trades off raw coverage against proximity/conciseness, is there a principled way to

pick λ for a given downstream FM, or is it genuinely a design knob left to the practitioner (tying back to your Results outline’s closing point)?

- Revisit the controllability/interpretability claim from Section 1’s contribution list: what does a reader actually get to inspect that a codebook-based tokenizer would not let them inspect? A short worked example (reusing the context-tree figure from Section 3.3) would make this concrete rather than asserted.
- *[NOTE: If the phenotype-discovery case study (`phenotype_clustering.py`, the IA/aneurysm cohort code currently live in `src/`) ends up in scope for this paper (see the Limitations note below), this is the natural place to discuss what that downstream task showed about the tokenization’s practical usefulness, as opposed to only its intrinsic metrics.]*

6 Limitations

The method inherits several limitations directly from its design.

Graph structure constraints. The context tree and the coverage score (Section 3) can only ever reflect the relationships that are actually encoded in the graph. Where SNOMED CT’s own definition of a concept is incomplete – a relevant attribute simply not modeled as an outgoing edge – the tokenizer cannot compensate for this: an under-specified concept will look under-specified in its context tree regardless of which token set T is chosen. **[DOUBT: “which our tokenizer can not do shit” in your original note – I read this as “our method has no way to fix or detect this,” which is what the sentence above says; flag if you meant something more specific, e.g. a particular class of missing relationships you’ve already run into.]**

Transductive selection. Token selection (Section 3.2) is transductive: it optimizes coverage for a fixed, known M , and a token set selected for one institution’s mapped concepts is not guaranteed to transfer to a different mapped set without re-running selection.

No external validation on a clinical task. **[DOUBT: This point is in tension with two other places in your outline: Method item 5 lists a “(maybe)” clinical-expert acceptability evaluation, and Future Work item 2 lists “external validation clinical task” as planned. As drafted here I’ve kept this limitation as stated (no external validation exists in this paper), but if the phenotype-discovery / IA-cohort case study (`phenotype_clustering.py`, still live in `src/` even though its driving notebooks sit in the now-obsolete `old_notebooks/`) is meant to be part of this paper, this limitation needs to be softened or removed rather than stated as-is – please confirm whether that case study is in scope.]**

Requires a DAG. The tokenizer’s core mechanism – IS_A transparency, non-IS_A subcontexts, and the coverage recursion’s reliance on well-defined finite hop distances – assumes an acyclic graph. It does not extend as-is to a

vocabulary with cycles, or to a general knowledge graph where the child/parent asymmetry of Section 2 does not hold.

7 Future Work

Two directions follow directly from the limitations above. First, where a mapping between SNOMED CT and another ontology exists, the same selection and tokenization mechanism could be applied across ontologies: tokens selected in one vocabulary could represent concepts from another, extending the interoperability argument from Section 1 to the token level itself. *[NOTE: This is the same point listed as "(maybe)" contribution item 5 in the Introduction – see the note there; keep it in only one of the two places.]* Second, addressing the transductive-selection limitation directly would mean validating the tokenization on a downstream clinical task external to the intrinsic metrics of Section 3.4.

[NOTE: Your existing PDF draft's own "Perspectives" section (Section 8) already sketches three further extensions, with citations, that don't appear anywhere in main.tex's Future Work outline. Pulling them in here since they seem like genuine future-work material rather than something to drop:]

Redundancy and token swaps. The greedy algorithm of Section 3.2 never removes a token once selected. A token can become nearly redundant once later tokens end up covering the same concepts it was originally selected for; a marginal-*loss*-based pass could identify and swap out such tokens after the fact.

Primitive vs. fully-defined concepts. Expanding a concept via its outgoing relationships (Section 3.3) implicitly assumes that doing so is lossless. For primitive SNOMED CT concepts – those not fully defined by their stated relationships – this may not hold. A reconstructibility factor $\alpha(u) \in [0, 1]$ ($\alpha(u) = 1$ for fully defined concepts, $\alpha(u) = \eta < 1$ for primitive ones) could discount the propagated coverage of Eq. (1) accordingly, while still treating a primitive concept selected *as its own token* as fully covered (coverage 1), since it is then preserved atomically rather than reconstructed from its relationships.

Combining with an ontology-similarity measure. An independently developed ontology-based similarity measure **[CITE: ontology-based similarity measure referenced in your PDF's Section 8 / reference [6]]** could evaluate how well semantic neighborhoods are preserved and how severe token collisions are, and could potentially be folded into the marginal-gain computation itself, e.g. as a facility-location-style term – provided the submodularity and monotonicity properties the greedy guarantee depends on (Section 3.2) are re-verified for the modified objective.